

Отчет по лабораторной работе № 23 по курсу алгоритмы и структуры данных

Студентка группы М8О-114БВ-25 Мозговая Жанна Евгеньевна, № по списку 13

Контакты www, e-mail, icq, skype mozgovay0607@gmail.com

Работа выполнена: «9» апреля 2026г.

Преподаватель: Сысоев Максим Алексеевич каф. 806

Входной контроль знаний с оценкой _____

Отчет сдан «9» апреля 2026 г., итоговая оценка _____

Подпись преподавателя _____

1. **Тема:** Динамические структуры данных. Деревья

2. **Цель работы:** Составить программу на языке Си в соответствии с вариантом.

3. **Задание (Вариант № 13):** проверить является ли дерево линейным списком вершин.

4. **Оборудование:**

Оборудование ПЭВМ студента, если использовалось:

Процессор __12th Gen Intel(R) Core(TM) i5-1235U, 1300 МГц, ядер: 10, логических процессоров:12.

Монитор __BOE PnP Monitor _____

5. **Программное обеспечение (лабораторное):**

Программное обеспечение ЭВМ студента, если использовалось:

Операционная система семейства Unix, наименование Ubuntu, версия 24.04 LTS, интерпретатор команд bash версия 5.2.21.

Система программирования: gcc (Ubuntu 11.4.0-1ubuntu1~22.04.2) версия 11.4

Редактор текстов: VS code версия: 1.107.1.

Язык программирования: C

6. **Идея, метод, алгоритм** решения задачи (в формах: словесной, псевдокода, графической [блок-схема, диаграмма, рисунок, таблица] или формальные спецификации с пред- и постусловиями)

Идея: проверить, является ли бинарное дерево поиска линейным списком вершин, то есть вырождено ли оно в односвязный список (каждый узел имеет не более одного потомка)

Метод:

рекурсивный обход дерева с проверкой условия: если у какого-либо узла существуют оба потомка (левый и правый), дерево не является линейным списком; если у всех узлов отсутствует хотя бы один из потомков — дерево линейно.

Алгоритм :

на вход подаётся корень дерева; базовый случай — пустое дерево возвращает истину; рекурсивно проверяются левое и правое поддеревья, при этом если у текущего узла есть оба потомка, функция сразу возвращает ложь; иначе рекурсивно вызывается для существующего потомка; результат — логическое значение (1 — дерево линейно, 0 — нет).

7. **Сценарий выполнения работы** [план работы, первоначальный текст программы в черновике (можно на отдельном листе) и тесты либо соображения по тестированию].

План работы:

- ✓ Понимание задачи
- ✓ Определение выходных данных: true/false
- ✓ Разработка концепции
- ✓ Проектирование алгоритма
- ✓ Написание черновой версии
- ✓ Тестирование и отладка
- ✓ Выявление и исправление ошибок
- ✓ Оформление кода

Первоначальный текст программы в черновике:

```
int main() {  
    TreeNode* root = NULL;  
    int choice;  
    char input[20];  
    Meme value;
```

```

do {
    printf("\n=== МЕНЮ ===\n");
    printf("1. Добавить узел\n");
    printf("2. Визуализировать дерево\n");
    printf("3. Удалить узел\n");
    printf("4. Проверить, является ли дерево линейным списком вершин\n");
    printf("5. Выйти\n");
    printf("Выбор: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1:
            printf("Введите значение (OMG, LOL, ROFL, LMAO, BRB, AFK): ");
            scanf("%s", input);
            value = stringToMeme(input);
            root = insert(root, value);
            break;
        case 2:
            printf("\nВизуализация дерева:\n");
            visualize(root, 0);
            break;
        case 3:
            printf("Введите значение для удаления: ");
            scanf("%s", input);
            value = stringToMeme(input);
            root = deleteNode(root, value);
            break;
        case 4:
            if (isLinearList(root)) {
                printf("ДА, дерево является линейным списком вершин.\n");
            } else {
                printf("НЕТ, дерево НЕ является линейным списком вершин.\n");
            }
            break;
        case 5:
            printf("До свидания!\n");
            break;
        default:
            printf("Неверный выбор!\n");
    }
} while (choice != 5);

freeTree(root);
return 0;
}

```

Тесты:

1. Выбор 4 (пустое дерево) → "ДА, дерево является линейным списком вершин."
2. Выбор 1 → OMG → Выбор 2 → визуализация: OMG → Выбор 4 → "ДА, дерево является линейным списком вершин."
3. Выбор 1 → OMG → Выбор 1 → LOL → Выбор 2 → визуализация: OMG (отступ 0), LOL (отступ 4) → Выбор 4 → "ДА, дерево является линейным списком вершин."
4. Выбор 1 → AFK → Выбор 1 → ROFL → Выбор 1 → LOL → Выбор 2 → визуализация: AFK, ROFL (отступ 4), LOL (отступ 8) → Выбор 4 → "ДА, дерево является линейным списком вершин."
5. Выбор 1 → AFK → Выбор 1 → ROFL → Выбор 1 → BRB → Выбор 1 → LOL → Выбор 2 → визуализация показывает узел ROFL с двумя потомками (LOL слева, BRB справа) → Выбор 4 → "НЕТ, дерево НЕ является линейным списком вершин."
6. Выбор 3 → ROFL → Выбор 2 → визуализация: AFK, BRB (правый потомок) → Выбор 4 → "ДА, дерево является линейным списком вершин."
7. Выбор 3 → XYZ → "Узел не найден."
8. Выбор 1 → OMG → Выбор 1 → omg → "Значение OMG уже есть в дереве!"
9. Выбор 1 → 123 → преобразуется в OMG (значение по умолчанию) → Выбор 2 → визуализация: OMG
10. Выбор 5 → "До свидания!" и выход из програм
- 11.

Пункты 1-7 отчета составляются строго до начала лабораторной работы.

Допущен к выполнению работы. Подпись преподавателя _____

8. Распечатка протокола (подклеить листинг окончательного варианта программы с тестовыми примерами, подписанный преподавателем).

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

typedef enum {
    OMG,
    LOL,
    ROFL,
    LMAO,
    BRB,
    AFK
} Meme;

const char* memeNames[] = {
    "OMG",
    "LOL",
    "ROFL",
    "LMAO",
    "BRB",
    "AFK"
};

const int MEME_COUNT = 6;

typedef struct TreeNode {
    Meme data;
    struct TreeNode* left;
    struct TreeNode* right;
} TreeNode;

TreeNode* createNode(Meme value) {
    TreeNode* newNode = (TreeNode*)malloc(sizeof(TreeNode));
    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

TreeNode* insert(TreeNode* root, Meme value) {
    if (root == NULL) {
        return createNode(value);
    }
    if (value < root->data) {
        root->left = insert(root->left, value);
    } else if (value > root->data) {
        root->right = insert(root->right, value);
    } else {
        printf("Значение %s уже есть в дереве!\n", memeNames[value]);
    }
    return root;
}

TreeNode* findMin(TreeNode* root) {
    while (root->left != NULL) {
        root = root->left;
    }
    return root;
}

TreeNode* deleteNode(TreeNode* root, Meme value) {
    if (root == NULL) {
        printf("Узел не найден.\n");
        return NULL;
    }
    if (value < root->data) {
```

```

    root->left = deleteNode(root->left, value);
} else if (value > root->data) {
    root->right = deleteNode(root->right, value);
} else {
    if (root->left == NULL) {
        TreeNode* temp = root->right;
        free(root);
        return temp;
    } else if (root->right == NULL) {
        TreeNode* temp = root->left;
        free(root);
        return temp;
    }

    TreeNode* temp = findMin(root->right);
    root->data = temp->data;
    root->right = deleteNode(root->right, temp->data);
}
return root;
}

```

```

void visualize(TreeNode* root, int depth) {
    if (root == NULL) return;

    visualize(root->right, depth + 1);

    for (int i = 0; i < depth; i++) {
        printf(" ");
    }
    printf("%s\n", memeNames[root->data]);

    visualize(root->left, depth + 1);
}

```

```

void freeTree(TreeNode* root) {
    if (root == NULL) return;
    freeTree(root->left);
    freeTree(root->right);
    free(root);
}

```

```

int isLinearList(TreeNode* root) {
    if (root == NULL) return 1;

    if (root->left != NULL && root->right != NULL) {
        return 0;
    }

    if (root->left != NULL) {
        return isLinearList(root->left);
    }
    if (root->right != NULL) {
        return isLinearList(root->right);
    }
    return 1;
}

```

```

Meme stringToMeme(char* str) {
    for (int i = 0; str[i]; i++) {
        str[i] = toupper(str[i]);
    }

    for (int i = 0; i < MEME_COUNT; i++) {
        if (strcmp(str, memeNames[i]) == 0) {
            return (Meme)i;
        }
    }
    return OMG;
}

```

```

int main() {
    TreeNode* root = NULL;
    int choice;
    char input[20];
    Meme value;

```

```

do {
    printf("\n=== МЕНЮ ===\n");
    printf("1. Добавить узел\n");
    printf("2. Визуализировать дерево\n");
    printf("3. Удалить узел\n");
    printf("4. Проверить, является ли дерево линейным списком вершин\n");
    printf("5. Выйти\n");
    printf("Выбор: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1:
            printf("Введите значение (OMG, LOL, ROFL, LMAO, BRB, AFK): ");
            scanf("%s", input);
            value = stringToMeme(input);
            root = insert(root, value);
            break;
        case 2:
            printf("\nВизуализация дерева:\n");
            visualize(root, 0);
            break;
        case 3:
            printf("Введите значение для удаления: ");
            scanf("%s", input);
            value = stringToMeme(input);
            root = deleteNode(root, value);
            break;
        case 4:
            if (isLinearList(root)) {
                printf("ДА, дерево является линейным списком вершин.\n");
            } else {
                printf("НЕТ, дерево НЕ является линейным списком вершин.\n");
            }
            break;
        case 5:
            printf("До свидания!\n");
            break;
        default:
            printf("Неверный выбор!\n");
    }
} while (choice != 5);

freeTree(root);
return 0;
}

```

9. Дневник отладки должен содержать дату и время сеансов отладки, и основные события(ошибки в сценарии и программе, нестандартные ситуации) и краткие комментарии к ним. В дневнике отладки приводятся сведения об использовании других ЭВМ, существенном участии преподавателя и других лиц в написании и отладке программы.

№	Лаб. или дом.	Дата	Время	Событие	Действие по исправлению	Примечание

10. Замечания автора по существу работы: замечания отсутствуют

11. Выводы: Программа реализует упорядоченное бинарное дерево поиска с узлами-перечислениями (Meme), поддерживает добавление, удаление, визуализацию с отступами и проверку, является ли дерево линейным списком вершин (то есть вырождено в список). Функция isLinearList рекурсивно проверяет, что ни один узел не имеет двух потомков. Тесты подтверждают корректную работу на всех режимах.

Подпись студента _____

